



PROCESADORES DE LENGUAJE - P7

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

1. Introducción

Esta práctica tiene como objetivo desarrollar un traductor de expresiones de notación infija a notación prefija utilizando Bison y Flex. A partir de una versión previa que convertía a notación postfija, se han realizado modificaciones para generar una salida compatible con el intérprete de Lisp.

El propósito de esta transformación es permitir la ejecución de expresiones en entornos funcionales como Lisp, donde la notación prefija es estándar. Se ha diseñado una gramática robusta que no solo convierte expresiones simples sino que también permite asignaciones y la impresión de resultados.

2. Descripción del Código

El archivo `calc7.y` contiene la gramática y las reglas semánticas para convertir expresiones en notación infija a notación prefija. Se han implementado las siguientes funcionalidades clave:

- **Manejo de operadores aritméticos:** Se soportan los operadores `+`, `-`, `*`, y `/`, respetando su precedencia y asociatividad.
- **Soporte para asignaciones:** Se pueden realizar asignaciones como `a = b + 2`, permitiendo la reutilización de variables.
- **Impresión de valores:** La sintaxis `@ expresion` se ha implementado para generar la salida `(print <expresion>)`, compatible con un entorno Lisp.
- **Almacenamiento de variables:** Se ha implementado una tabla de símbolos para mantener los valores de variables asignadas y permitir su reutilización en expresiones posteriores.
- **Gestión eficiente de memoria:** Se han añadido mecanismos para evitar fugas de memoria y optimizar el uso de recursos dinámicos.

La conversión de notación se realiza almacenando la salida en `$$cadena`, lo que permite una construcción estructurada y modular del resultado prefijo, garantizando que solo se imprima en el axioma para mantener la consistencia de la traducción.

3. Gestión de Memoria

Dado que el procesamiento de expresiones requiere la asignación dinámica de memoria, se han implementado estrategias para garantizar un uso eficiente y evitar fugas:

- **Uso de `malloc()` y `free()`:** Se utiliza `malloc()` en `generate_string()` para reservar memoria para cada expresión generada dinámicamente.
- **Liberación de memoria inmediata:** Se implementó la función `free_memory()` para liberar cadenas que ya no se necesitan, evitando el crecimiento innecesario de la memoria utilizada.
- **Gestión de la tabla de símbolos:** La tabla de símbolos almacena las variables definidas y se asegura de liberar memoria cuando estas son reasignadas o al finalizar la ejecución.

Gracias a estas estrategias, el programa mantiene un consumo de memoria controlado incluso con múltiples asignaciones y operaciones anidadas.

4. Casos de Prueba

Para validar la funcionalidad del traductor, se han ejecutado diversos casos de prueba, asegurando que las expresiones se traduzcan correctamente y respeten las reglas sintácticas y semánticas establecidas.

Expresiones Aritméticas

Unset

$5 + 3 \rightarrow (+\ 5\ 3)$

$7 - 2 \rightarrow (-\ 7\ 2)$

$8 * 4 \rightarrow (*\ 8\ 4)$

$9 / 3 \rightarrow (/ \ 9\ 3)$

Expresiones con Diferente Precedencia

Unset

$5 + 3 * 2 \rightarrow (+\ 5\ (*\ 3\ 2))$

$(5 + 3) * 2 \rightarrow (*\ (+\ 5\ 3)\ 2)$

$10 - 4 / 2 \rightarrow (-\ 10\ (/ \ 4\ 2))$

$(10 - 4) / 2 \rightarrow (/ \ (-\ 10\ 4)\ 2)$

Asignaciones de Variables

Unset

$a = 5 + 3 \rightarrow (= \ a\ (+\ 5\ 3))$

$b = a * 2 \rightarrow (= \ b\ (*\ a\ 2))$

$c = (b - 4) / 2 \rightarrow (= \ c\ (/ \ (-\ b\ 4)\ 2))$

Uso del operador @ para imprimir valores

Unset

$@\ (5 + 3) \rightarrow (\text{print}\ (+\ 5\ 3))$

$@\ a \rightarrow (\text{print}\ a)$

5. Conclusiones

El traductor desarrollado ha demostrado ser capaz de convertir expresiones infijas en notación prefija de manera eficiente, respetando la jerarquía de operaciones y permitiendo el uso de variables y la impresión de resultados. Durante el desarrollo, se abordaron y resolvieron varios desafíos, entre ellos:

- **Manejo de memoria eficiente:** Se logró optimizar el uso de memoria, asegurando la correcta asignación y liberación de recursos.
- **Corrección de errores en la gramática:** Se ajustaron las reglas de precedencia y asociatividad para garantizar la correcta conversión de expresiones anidadas.
- **Compatibilidad con el intérprete Lisp:** La salida generada es compatible con entornos funcionales como Common Lisp, lo que permite la evaluación directa de las expresiones traducidas.